

Experiment analysis step 1: populations and canonical datasets

Participant flow, attrition/compliance, and analysis-ready exports

W241 Plank Study

2026-04-15

Contents

0.1	Purpose	1
0.2	Load inputs	1
0.3	Reading the flow correctly	2
0.4	Build participant flow dataset	2
0.5	Participant flow summaries	3
0.6	Attrition-bias check	5
0.7	Build paired analytic dataset	9
0.8	Build long repeated-measures dataset	10
0.9	Write derived outputs	10
0.10	Verification	11

0.1 Purpose

This document creates the first set of experiment-analysis objects:

1. analysis/derived/participant_flow.csv
2. analysis/derived/paired_analysis.csv
3. analysis/derived/long_repeated_measures.csv

It uses:

- w241_experiment_wide.csv for observed sessions, condition-level outcomes, and survey/process variables.
- W241 Plank Study Data - participants.csv for the full registered participant roster.

0.2 Load inputs

```
participants <- read_participants()
wide <- read_experiment_wide()

stopifnot(uniqueN(participants$participant_id) == nrow(participants))
stopifnot(uniqueN(wide$participant_id) == nrow(wide))
stopifnot(all(wide$participant_id %in% participants$participant_id))

participants[, assigned_order := fcase(
  group_label == "trancepop", "high_tempo_first",
  group_label == "poptrance", "low_tempo_first",
  default = NA_character_
)]
```

0.3 Reading the flow correctly

This step intentionally starts from the **full initial distribution** and only then narrows to the analytic samples.

- First, we show the full registered roster and how many participants reach each observation stage.
- Next, we separate **attrition** (for example, not returning for session 2) from **compliance / protocol exclusions**.
- Only after those counts are visible do we define the **primary analytic sample** and the **first-period benchmark sample**.

This mirrors the report plan: the reader should see the whole study population first, then exactly why some participants are not used for a given estimand.

0.4 Build participant flow dataset

The participant roster is the base denominator for flow accounting. We left-join the experiment wide file onto the full participant list so registered participants with zero observed sessions still appear in the flow dataset.

```
flow <- merge(
  participants,
  wide,
  by = "participant_id",
  all.x = TRUE,
  suffixes = c("_participant", ""))

flow[, registered := 1L]
flow[, randomized := as.integer(!is.na(assigned_order))]
flow[, order_observed := as.integer(!is.na(crossover_order))]
flow[, period1_observed := as.integer(!is.na(plank_s1))]
flow[, period2_observed := as.integer(!is.na(plank_s2))]
flow[, completed_both_observed := as.integer(!is.na(plank_s1) & !is.na(plank_s2))]
flow[, attrited_before_session2 := as.integer(period1_observed == 1L & period2_observed == 0L)]

flow[, missing_primary_condition := as.integer(is.na(plank_high_tempo) | is.na(plank_low_tempo))]
flow[, impossible_time := as.integer(
  (!is.na(plank_high_tempo) & plank_high_tempo > 600) |
  (!is.na(plank_low_tempo) & plank_low_tempo > 600))
]]
flow[, washout_violation := as.integer(
  completed_both_observed == 1L &
  !is.na(elapsed_sessions_hours) &
  elapsed_sessions_hours < 24
)]

flow[, in_primary_analysis := as.integer(
  completed_both_observed == 1L &
  missing_primary_condition == 0L &
  impossible_time == 0L &
  washout_violation == 0L
)]

flow[, in_first_period_benchmark := as.integer(
  randomized == 1L &
  period1_observed == 1L &
```

```

    !is.na(plank_s1) &
    plank_s1 <= 600
  ])

flow[, primary_exclusion_reason := fcase(
  in_primary_analysis == 1L, NA_character_,
  missing_primary_condition == 1L, "missing one condition",
  impossible_time == 1L, "impossible time > 600s",
  washout_violation == 1L, "washout < 24h",
  default = "other"
)]

flow[, benchmark_exclusion_reason := fcase(
  in_first_period_benchmark == 1L, NA_character_,
  randomized == 0L, "not randomized",
  period1_observed == 0L, "missing session 1 outcome",
  !is.na(plank_s1) & plank_s1 > 600, "impossible session 1 time > 600s",
  default = "other"
)]

setorder(flow, participant_registered_at, participant_id)

```

0.5 Participant flow summaries

```

flow_table <- data.table(
  population = c(
    "Registered participants",
    "Randomized order assigned",
    "Session 1 observed",
    "Session 2 observed",
    "Completed both sessions",
    "Primary analytic sample",
    "First-period benchmark sample"
  ),
  n = c(
    nrow(flow),
    flow[, sum(randomized)],
    flow[, sum(period1_observed)],
    flow[, sum(period2_observed)],
    flow[, sum(completed_both_observed)],
    flow[, sum(in_primary_analysis)],
    flow[, sum(in_first_period_benchmark)]
  )
)

kable(flow_table, caption = "Table A. Participant flow")

```

Table 1: Table A. Participant flow

population	n
Registered participants	58
Randomized order assigned	58

population	n
Session 1 observed	47
Session 2 observed	40
Completed both sessions	40
Primary analytic sample	38
First-period benchmark sample	46

```

flow_nodes <- data.table(
  id = c("registered", "randomized", "session1", "session2", "completed", "primary", "benchmark", "excl
  x = c(1.2, 3.6, 6.0, 8.4, 10.8, 13.2, 6.0, 10.8),
  y = c(3.1, 3.1, 3.1, 3.1, 3.1, 3.1, 0.95, 5.25),
  label = c(
    paste0("Registered\nn = ", nrow(flow)),
    paste0("Randomized\nn = ", flow[, sum(randomized)]),
    paste0("Session 1 observed\nn = ", flow[, sum(period1_observed)]),
    paste0("Session 2 observed\nn = ", flow[, sum(period2_observed)]),
    paste0("Completed both\nn = ", flow[, sum(completed_both_observed)]),
    paste0("Primary analytic sample\nn = ", flow[, sum(in_primary_analysis)]),
    paste0("First-period benchmark\nn = ", flow[, sum(in_first_period_benchmark)]),
    paste0(
      "Excluded from primary\nn = ", flow[, sum(completed_both_observed)] - flow[, sum(in_primary_analy
      "\n1 impossible time > 600s\n1 washout < 24h"
    )
  )
)

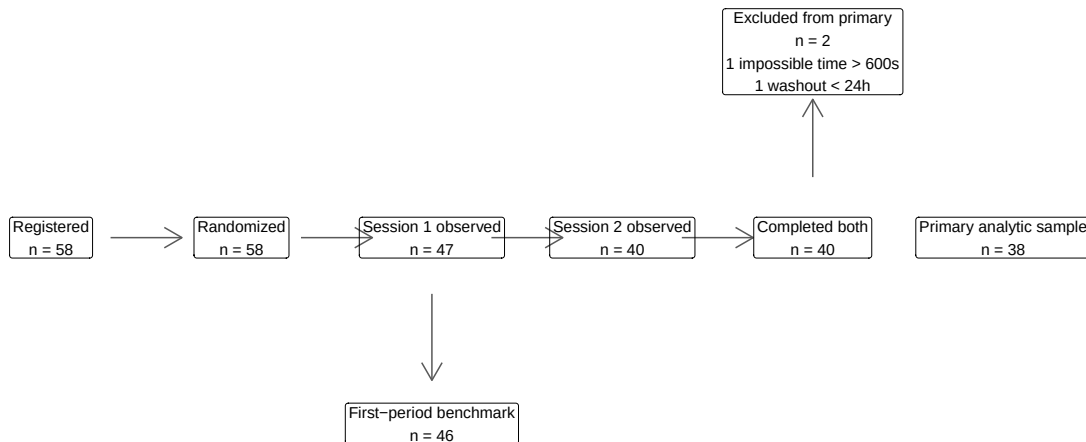
flow_edges <- data.table(
  x = c(1.95, 4.35, 6.75, 9.15, 6.0, 10.8),
  y = c(3.1, 3.1, 3.1, 3.1, 2.45, 3.8),
  xend = c(2.85, 5.25, 7.65, 10.05, 6.0, 10.8),
  yend = c(3.1, 3.1, 3.1, 3.1, 1.45, 4.7)
)

ggplot() +
  geom_segment(
    data = flow_edges,
    aes(x = x, y = y, xend = xend, yend = yend),
    arrow = grid::arrow(length = grid::unit(0.18, "inches")),
    linewidth = 0.45,
    color = "gray35"
  ) +
  geom_label(
    data = flow_nodes,
    aes(x = x, y = y, label = label),
    label.size = 0.25,
    size = 3.5,
    label.padding = grid::unit(0.18, "lines")
  ) +
  coord_cartesian(xlim = c(0.2, 14.5), ylim = c(0.3, 5.9), clip = "off") +
  labs(
    title = "Participant flow from registration to analytic samples",
    subtitle = "Full distribution first, then attrition and protocol-based exclusions"
  )

```

```
) +
  theme_void(base_size = 12)
## Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0.
## i Please use the `linewidth` argument instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Participant flow from registration to analytic samples
Full distribution first, then attrition and protocol-based exclusions



```
attrition_by_order <- flow[
  randomized == 1L,
  .(
    started_session1 = sum(period1_observed),
    dropped_before_session2 = sum(attrited_before_session2),
    retained_completed_both = sum(completed_both_observed)
  ),
  by = assigned_order
][order(assigned_order)]

kable(attrition_by_order, caption = "Table B. Attrition by randomized order")
```

Table 2: Table B. Attrition by randomized order

assigned_order	started_session1	dropped_before_session2	retained_completed_both
high_tempo_first	23	4	19
low_tempo_first	24	3	21

0.6 Attrition-bias check

This check asks whether retention through both observed sessions differed meaningfully by randomized order arm or baseline demographics among randomized participants. It is intentionally light-touch: the goal is to surface obvious selection patterns without overfitting a small attrition model.

```
rand_flow <- copy(flow[randomized == 1L])
rand_flow[, retained_completed_both := as.integer(completed_both_observed == 1L)]
```

```

rand_flow[, baseline_age := coalesce_num(participant_age, age)]
rand_flow[, baseline_gender := tolower(trimws(coalesce_chr(participant_gender, gender)))]
rand_flow[, male_indicator := as.integer(baseline_gender == "male")]

safe_fisher_p <- function(x, y) {
  tmp <- data.table(x = x, y = y)[!is.na(x) & !is.na(y)]
  if (nrow(tmp) == 0L || uniqueN(tmp$x) < 2L || uniqueN(tmp$y) < 2L) {
    return(NA_real_)
  }
  fisher.test(table(tmp$x, tmp$y))$p.value
}

safe_t_p <- function(value, group) {
  tmp <- data.table(value = value, group = group)[!is.na(value) & !is.na(group)]
  if (nrow(tmp) == 0L || uniqueN(tmp$group) < 2L) {
    return(NA_real_)
  }
  t.test(value ~ group, data = tmp)$p.value
}

retention_demo <- rand_flow[
  ,
  .(
    n = .N,
    mean_age = mean(baseline_age, na.rm = TRUE),
    sd_age = sd(baseline_age, na.rm = TRUE),
    share_male = mean(male_indicator, na.rm = TRUE)
  ),
  by = retained_completed_both
][order(-retained_completed_both)]

retention_demo[, retained_group := fifelse(
  retained_completed_both == 1L,
  "Completed both sessions",
  "Did not complete both"
)]
retention_demo[, `:=`(
  mean_age = round(mean_age, 2),
  sd_age = round(sd_age, 2),
  share_male = round(100 * share_male, 1)
)]
retention_demo <- retention_demo[, .(retained_group, n, mean_age, sd_age, share_male)]

attrition_bias_by_order <- rand_flow[
  ,
  .(
    randomized_participants = .N,
    retained_completed_both = sum(retained_completed_both),
    not_retained = sum(1L - retained_completed_both),
    retention_rate_pct = 100 * mean(retained_completed_both)
  ),
  by = assigned_order
][order(assigned_order)]

```

```

attrition_bias_by_order[, retention_rate_pct := round(retention_rate_pct, 1)]

gender_test_dt <- copy(rand_flow)[baseline_gender %in% c("male", "female")]
gender_test_dt[, retained_group := fifelse(
  retained_completed_both == 1L,
  "Completed both sessions",
  "Did not complete both"
)]

gender_counts <- gender_test_dt[,
  .N,
  by = .(baseline_gender, retained_group)
]
gender_matrix <- xtabs(N ~ baseline_gender + retained_group, data = gender_counts)

gender_balance_tab <- data.table(
  retained_group = colnames(gender_matrix),
  male_n = as.integer(gender_matrix["male", ]),
  female_n = as.integer(gender_matrix["female", ]),
  male_share_pct = round(100 * gender_matrix["male", ] / colSums(gender_matrix), 1)
)

expected_gender_counts <- suppressWarnings(chisq.test(gender_matrix, correct = FALSE)$expected)
gender_p_value <- if (any(expected_gender_counts < 5)) {
  fisher.test(gender_matrix)$p.value
} else {
  prop.test(
    x = gender_matrix["male", ],
    n = colSums(gender_matrix),
    correct = FALSE
  )$p.value
}

attrition_bias_tests <- data.table(
  comparison = c(
    "Retention by randomized order",
    "Baseline age by retention status",
    "Baseline gender by retention status"
  ),
  p_value = c(
    safe_fisher_p(rand_flow$assigned_order, rand_flow$retained_completed_both),
    safe_t_p(rand_flow$baseline_age, rand_flow$retained_completed_both),
    gender_p_value
  )
)
attrition_bias_tests[, p_value := round(p_value, 4)]

kable(retention_demo, caption = "Table C. Baseline demographics by retention status")

```

Table 3: Table C. Baseline demographics by retention status

retained_group	n	mean_age	sd_age	share_male
Completed both sessions	40	36.30	13.15	70.0
Did not complete both	18	32.67	10.05	66.7

```
kable(attrition_bias_by_order, caption = "Table D. Retention by randomized order")
```

Table 4: Table D. Retention by randomized order

assigned_order	randomized_participants	retained_completed_both	not_retained	retention_rate_pct
high_tempo_first	30	19	11	63.3
low_tempo_first	28	21	7	75.0

```
kable(gender_balance_tab, caption = "Table E. Baseline gender counts by retention status")
```

Table 5: Table E. Baseline gender counts by retention status

retained_group	male_n	female_n	male_share_pct
Completed both sessions	28	12	70.0
Did not complete both	12	6	66.7

```
kable(attrition_bias_tests, caption = "Table F. Attrition-bias diagnostic p-values")
```

Table 6: Table F. Attrition-bias diagnostic p-values

comparison	p_value
Retention by randomized order	0.4022
Baseline age by retention status	0.2556
Baseline gender by retention status	0.7996

These diagnostics should be read as descriptive checks rather than proof that attrition was ignorable. They focus on retention through Session 2 among randomized participants and cannot rule out selection on unobserved factors. They are also conceptually separate from the protocol-based primary-analysis exclusions, which apply only after a participant completed both observed sessions.

```
compliance_table <- data.table(
  issue = c(
    "Impossible time > 600s",
    "Washout violation < 24h",
    "Missing one condition",
    "Excluded from primary analysis"
  ),
  n = c(
    flow[, sum(impossible_time)],
    flow[, sum(washout_violation)],
    flow[, sum(missing_primary_condition)],
    flow[, sum(in_primary_analysis == 0L & completed_both_observed == 1L)]
)
```

```
)
```

```
kable(compliance_table, caption = "Table G. Compliance and exclusion reasons")
```

Table 7: Table G. Compliance and exclusion reasons

issue	n
Impossible time > 600s	1
Washout violation < 24h	1
Missing one condition	18
Excluded from primary analysis	2

0.7 Build paired analytic dataset

The paired dataset keeps one row per primary analytic participant and standardizes condition-level secondary outcomes so later Rmds can work in `high_tempo` / `low_tempo` terms directly.

```
paired <- flow[
  in_primary_analysis == 1L,
  .(
    participant_id,
    crossover_order,
    first_session_is_high_tempo,
    participant_age,
    participant_gender,
    participant_locale,
    participant_group_index,
    participant_group_label,
    participant_registered_at,
    participant_session1_planned_at,
    participant_session2_planned_at,
    elapsed_sessions_hours,
    first_period_condition = audio_s1,
    second_period_condition = audio_s2,
    first_period_tte = plank_s1,
    second_period_tte = plank_s2,
    high_tempo_tte = plank_high_tempo,
    low_tempo_tte = plank_low_tempo,
    diff_high_minus_low,
    energy_high_tempo = condition_from_sessions(audio_s1, audio_s2, pre_energy_pre_plank_s1, pre_energy_
    energy_low_tempo = condition_from_sessions(audio_s1, audio_s2, pre_energy_pre_plank_s1, pre_energy_
    motivation_high_tempo = condition_from_sessions(audio_s1, audio_s2, pre_motivation_pre_plank_s1, pre_
    motivation_low_tempo = condition_from_sessions(audio_s1, audio_s2, pre_motivation_pre_plank_s1, pre_
    music_liking_high_tempo = condition_from_sessions(audio_s1, audio_s2, pre_music_liking_s1, pre_musi
    music_liking_low_tempo = condition_from_sessions(audio_s1, audio_s2, pre_music_liking_s1, pre_musi
    rpe_high_tempo = condition_from_sessions(audio_s1, audio_s2, post_rpe_s1, post_rpe_s2, "high_tempo"),
    rpe_low_tempo = condition_from_sessions(audio_s1, audio_s2, post_rpe_s1, post_rpe_s2, "low_tempo"),
    headphones_high_tempo = condition_from_sessions(audio_s1, audio_s2, post_headphones_s1, post_headpho
    headphones_low_tempo = condition_from_sessions(audio_s1, audio_s2, post_headphones_s1, post_headpho
    pause_high_tempo = condition_from_sessions(audio_s1, audio_s2, post_plank_pause_s1, post_plank_paus
    pause_low_tempo = condition_from_sessions(audio_s1, audio_s2, post_plank_pause_s1, post_plank_paus
    volume_clear_high_tempo = condition_from_sessions(audio_s1, audio_s2, post_volume_clear_s1, post_vo
    volume_clear_low_tempo = condition_from_sessions(audio_s1, audio_s2, post_volume_clear_s1, post_vo
```

```

    activity_level_baseline = pre_activity_level_s1,
    activity_type_baseline = pre_activity_type_s1,
    plank_frequency_baseline = pre_plank_frequency_s1
  )
]

stopifnot(uniqueN(paired$participant_id) == nrow(paired))

```

0.8 Build long repeated-measures dataset

The long dataset keeps one row per observed session and carries analysis flags so later model files can subset consistently without re-implementing the exclusion logic.

```

make_period_dt <- function(base, sn) {
  audio_col <- paste0("audio_s", sn)
  plank_col <- paste0("plank_s", sn)
  out <- data.table(
    participant_id = base$participant_id,
    period = sn,
    treatment = base[[audio_col]],
    high_tempo = as.integer(base[[audio_col]] == "high_tempo"),
    crossover_order = base$crossover_order,
    first_session_is_high_tempo = base$first_session_is_high_tempo,
    outcome_tte = base[[plank_col]],
    pre_energy = base[[paste0("pre_energy_pre_plank_s", sn)]],
    pre_motivation = base[[paste0("pre_motivation_pre_plank_s", sn)]],
    pre_music_liking = base[[paste0("pre_music_liking_s", sn)]],
    post_rpe = base[[paste0("post_rpe_s", sn)]],
    post_headphones = base[[paste0("post_headphones_s", sn)]],
    post_plank_pause = base[[paste0("post_plank_pause_s", sn)]],
    post_volume_clear = base[[paste0("post_volume_clear_s", sn)]],
    participant_age = base$participant_age,
    participant_gender = base$participant_gender,
    participant_locale = base$participant_locale,
    activity_level_baseline = base$pre_activity_level_s1,
    activity_type_baseline = base$pre_activity_type_s1,
    plank_frequency_baseline = base$pre_plank_frequency_s1,
    impossible_time = base$impossible_time,
    washout_violation = base$washout_violation,
    in_primary_analysis = base$in_primary_analysis,
    in_first_period_benchmark = base$in_first_period_benchmark
  )
  out[!is.na(out$treatment) & !is.na(out$outcome_tte)]
}

long_dt <- rbindlist(list(make_period_dt(flow, 1L), make_period_dt(flow, 2L)), fill = TRUE)
setorder(long_dt, participant_id, period)

stopifnot(all(long_dt$period %in% c(1L, 2L)))

```

0.9 Write derived outputs

```

participant_flow <- flow[, .(
  participant_id,
  participant_age = age,
  participant_gender = gender,
  participant_locale = locale,
  participant_group_index = group_index,
  participant_group_label = group_label,
  participant_sessions_completed = sessions_completed,
  participant_registered_at = registered_at,
  participant_session1_planned_at = session1_planned_at,
  participant_session2_planned_at = session2_planned_at,
  assigned_order,
  registered,
  randomized,
  order_observed,
  period1_observed,
  period2_observed,
  completed_both_observed,
  attrited_before_session2,
  missing_primary_condition,
  impossible_time,
  washout_violation,
  in_primary_analysis,
  in_first_period_benchmark,
  primary_exclusion_reason,
  benchmark_exclusion_reason
)]

write_derived_csv(participant_flow, "participant_flow.csv")
write_derived_csv(paired, "paired_analysis.csv")
write_derived_csv(long_dt, "long_repeated_measures.csv")

```

0.10 Verification

```

pf <- fread(derived_path("participant_flow.csv"))
pd <- fread(derived_path("paired_analysis.csv"))
ld <- fread(derived_path("long_repeated_measures.csv"))

stopifnot(nrow(pf) == nrow(participants))
stopifnot(all(wide$participant_id %in% pf$participant_id))
stopifnot(nrow(pd) == flow[, sum(in_primary_analysis)])
stopifnot(uniqueN(pd$participant_id) == nrow(pd))
stopifnot(nrow(ld) == flow[, sum(period1_observed + period2_observed)])

message("Wrote ", nrow(pf), " rows to ", derived_path("participant_flow.csv"))
## Wrote 58 rows to /home/user/MIDS/w241/analysis/derived/participant_flow.csv
message("Wrote ", nrow(pd), " rows to ", derived_path("paired_analysis.csv"))
## Wrote 38 rows to /home/user/MIDS/w241/analysis/derived/paired_analysis.csv
message("Wrote ", nrow(ld), " rows to ", derived_path("long_repeated_measures.csv"))
## Wrote 87 rows to /home/user/MIDS/w241/analysis/derived/long_repeated_measures.csv

```